Ex. No.	5	**Name**	Ajmal Rahman A
Date	18-10-2026	**Reg. No.**	192324094
Sub. Name	Deep Learning	**Sub. Code**	CSA4702
Semester	MS2 September 2026		

EXERCISE NAME

BIAS–VARIANCE EVALUATION OF REAL-ESTATE PRICE MODELS

SCENARIO

A real-estate price prediction system compares a linear regression model, a decision tree and a high-complexity polynomial model. Evaluate their training and testing errors to identify which model underfits, which overfits and which is the most suitable for predicting the price of a house.

ALGORITHM

Step 1 : Import the packages used in this exercise — Scikit-learn for the three models, the train–test split, the cross-validation and the evaluation metrics, NumPy for the array algebra, Pandas for the tabular handling of the records and Matplotlib for the plots.

Step 2 : Build the dataset of 300 houses, in which the three features are the built-up area, the number of bedrooms and the age of the house, and the target is the price in rupees lakh. Generate the price with a deliberately nonlinear dependence on the area and on the age, together with a random market noise, so that no straight line can fit it exactly.

Step 3 : Compute the descriptive statistics of the three features and of the price.

Step 4 : Split the data into a training set of 225 houses and a testing set of 75 houses, so that every model is judged on houses it has never seen.

Step 5 : Define the three competing models — a linear regression of low complexity, an unrestricted decision tree that is allowed to grow until every leaf is pure, and a polynomial regression of degree 9 of very high complexity. Standardise the features inside a pipeline so that the polynomial terms stay numerically stable.

Step 6 : Fit every model on the training set and compute the root mean squared error, the mean absolute error and the R^2 score on the training set and on the testing set.

Step 7 : Diagnose the behaviour of every model from the two errors — an error that stays high on both sets indicates underfitting, that is high bias; an error near zero on the training set together with a far larger error on the testing set indicates overfitting, that is high variance; and errors of similar size on both sets indicate a balanced fit.

Step 8 : Confirm the diagnosis with five-fold cross-validation, in which every model is trained and tested five times on different folds of the data, and report the mean and the standard deviation of the fold errors; a large standard deviation is itself a sign of high variance.

Step 9 : Study the trade-off directly by fitting polynomial models of degree 1 to 9 and recording the training and testing error of each, so that the point at which the testing error begins to rise while the training error keeps falling can be located.

Step 10 : Plot the training and testing errors of the three models as paired bars, and plot the error against the model complexity.

Step 11 : Select the most suitable model as the one with the lowest testing error and the smallest gap between its training and testing errors. Stop.

SOURCE CODE

Module 1 : Importing the Required Packages

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

plt.rcParams["font.family"]          = "serif"
plt.rcParams["font.serif"]           = ["Cambria", "Caladea", "DejaVu Serif"]
plt.rcParams["axes.spines.top"]      = False
plt.rcParams["axes.spines.right"]    = False
plt.rcParams["legend.frameon"]       = False
```

Module 2 : Building the Real-Estate Dataset

```
np.random.seed(42)
N = 300                                              # number of houses

area     = np.clip(np.random.normal(1450, 430, N), 500, 3200)      # square feet
bedrooms = np.clip(np.round(1 + area / 650 + np.random.normal(0, 0.6, N)), 1, 6)
age      = np.clip(np.random.gamma(2.6, 4.2, N), 0, 45)            # years

price = (6.0
         + 0.0062 * area                             # linear growth with area
         - 0.00000085 * (area - 1200) ** 2     # diminishing returns (nonlinear)
         + 1.15 * bedrooms
         - 0.22 * age + 0.0028 * age ** 2            # nonlinear effect of age
         + np.random.normal(0, 1.5, N))              # market noise
price = np.round(np.clip(price, 3, 60), 2)           # price in Rs lakh

df = pd.DataFrame({"House ID"       : ["H%03d" % (i + 1) for i in range(N)],
                   "Area (sq ft)"   : np.round(area, 1),
                   "Bedrooms"       : bedrooms.astype(int),
                   "Age (years)"    : np.round(age, 1),
                   "Price (Rs lakh)": price})

FEATURES = ["Area (sq ft)", "Bedrooms", "Age (years)"]
TARGET   = "Price (Rs lakh)"

X = df[FEATURES].to_numpy(dtype=float)
y = df[TARGET].to_numpy(dtype=float)
print("Feature matrix X :", X.shape, "  Target y :", y.shape)
print(df.head(10).to_string(index=False))
```

Module 3 : Descriptive Statistics

```
desc = df[FEATURES + [TARGET]].describe().round(2)
print(desc)
```

Module 4 : Train-Test Split

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=42)
print("Training houses :", X_tr.shape[0], "  Testing houses :", X_te.shape[0])
```

Module 5 : Defining the Three Competing Models

```
models = {
    "Linear Regression"            : make_pipeline(StandardScaler(),
                                                   LinearRegression()),
    "Decision Tree (unrestricted)" : DecisionTreeRegressor(random_state=42),
    "Polynomial Regression (deg 9)": make_pipeline(
        StandardScaler(), PolynomialFeatures(degree=9, include_bias=False),
        LinearRegression()),
}
for name, mdl in models.items():
    mdl.fit(X_tr, y_tr)
    print("Fitted :", name)
```

Module 6 : Training and Testing Errors of every Model

```
def diagnose(tr_rmse, te_rmse, tr_r2, te_r2):
    if te_r2 < 0:                                  # prediction worse than the mean
        return "Severe overfitting (very high variance)"
    if te_rmse > 1.4 * tr_rmse or (tr_r2 - te_r2) > 0.15:
        return "Overfitting (high variance)"
    if tr_r2 < 0.88:                                # error stays high on both sets
        return "Slight underfitting (high bias, low variance)"
    return "Good fit (balanced bias and variance)"

results = []
for name, mdl in models.items():
    p_tr, p_te = mdl.predict(X_tr), mdl.predict(X_te)
    tr_rmse = np.sqrt(mean_squared_error(y_tr, p_tr))
    te_rmse = np.sqrt(mean_squared_error(y_te, p_te))
    tr_r2, te_r2 = r2_score(y_tr, p_tr), r2_score(y_te, p_te)
    results.append({
        "name": name, "tr_rmse": tr_rmse, "te_rmse": te_rmse,
        "tr_mae": mean_absolute_error(y_tr, p_tr),
        "te_mae": mean_absolute_error(y_te, p_te),
        "tr_r2": tr_r2, "te_r2": te_r2, "gap": te_rmse - tr_rmse,
        "verdict": diagnose(tr_rmse, te_rmse, tr_r2, te_r2), "p_te": p_te})
    print("%-30s train RMSE %6.3f | test RMSE %6.3f | R2 %.4f / %.4f | %s"
          % (name, tr_rmse, te_rmse, tr_r2, te_r2, results[-1]["verdict"]))

best = min(results, key=lambda r: r["te_rmse"])
print("Most suitable model :", best["name"])
```

Module 7 : Five-Fold Cross-Validation of the Three Models

```
kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv = {}
for name, mdl in models.items():
    sc = -cross_val_score(mdl, X, y, cv=kf,
                          scoring="neg_root_mean_squared_error")
    cv[name] = sc
    print("%-30s fold RMSE %s | mean %.3f | std %.3f"
          % (name, np.round(sc, 3), sc.mean(), sc.std()))
```

Module 8 : Bias-Variance Behaviour across Polynomial Degrees

```
degrees   = range(1, 10)
deg_train, deg_test = [], []
for d in degrees:
    mdl = make_pipeline(StandardScaler(),
                        PolynomialFeatures(degree=d, include_bias=False),
```

```
                        LinearRegression()).fit(X_tr, y_tr)
    deg_train.append(np.sqrt(mean_squared_error(y_tr, mdl.predict(X_tr))))
    deg_test.append(np.sqrt(mean_squared_error(y_te, mdl.predict(X_te))))
    print("degree %d | train RMSE %7.4f | test RMSE %9.4f"
          % (d, deg_train[-1], deg_test[-1]))
```

Module 9 : Visualisation of the Model Comparison

```
labels = ["Linear\nRegression", "Decision Tree\n(unrestricted)",
          "Polynomial\n(degree 9)"]
pos    = np.arange(3)

fig, ax = plt.subplots(figsize=(6.6, 3.4))
ax.bar(pos - 0.2, [r["tr_rmse"] for r in results], width=0.4,
       color="#2F6DB5", label="Training RMSE")
ax.bar(pos + 0.2, [r["te_rmse"] for r in results], width=0.4,
       color="#C0702A", label="Testing RMSE")
for i, r in enumerate(results):
    ax.text(i - 0.2, r["tr_rmse"] + 0.06, "%.2f" % r["tr_rmse"],
            ha="center", fontsize=8)
    ax.text(i + 0.2, r["te_rmse"] + 0.06, "%.2f" % r["te_rmse"],
            ha="center", fontsize=8)
ax.set_xticks(pos, labels, fontsize=9)
ax.set_ylabel("Root mean squared error (Rs lakh)")
ax.grid(axis="y", alpha=0.55);  ax.set_axisbelow(True)
ax.legend(loc="upper left", fontsize=8)
fig.tight_layout()
fig.savefig("plot1_model_errors.png", dpi=300, bbox_inches="tight")

fig, ax = plt.subplots(figsize=(6.6, 3.4))
ax.plot(list(degrees), deg_train, "o-", color="#2F6DB5", lw=1.6, ms=4,
        label="Training RMSE")
ax.plot(list(degrees), deg_test,  "s--", color="#C0702A", lw=1.6, ms=4,
        label="Testing RMSE")
ax.set_yscale("log")
ax.set_xlabel("Degree of the polynomial model (model complexity)")
ax.set_ylabel("RMSE (Rs lakh), log scale")
ax.set_xticks(list(degrees))
ax.axvspan(0.6, 1.4,  color="#2F6DB5", alpha=0.07)
ax.axvspan(6.6, 9.4,  color="#C0702A", alpha=0.07)
ax.text(1.0, max(deg_test)*0.4, "underfitting", fontsize=8,
        ha="center", color="#2F6DB5")
ax.text(8.0, max(deg_test)*0.4, "overfitting",  fontsize=8,
        ha="center", color="#C0702A")
ax.grid(alpha=0.55);  ax.set_axisbelow(True)
ax.legend(loc="upper center", fontsize=8)
fig.tight_layout()
fig.savefig("plot2_complexity_curve.png", dpi=300, bbox_inches="tight")
```

OUTPUT

Table 1 - Real-Estate Dataset Sample (First 10 Houses)

S.No	House ID	Area (sq ft)	Bedrooms	Age (years)	Price (Rs lakh)
1	H001	1663.6	3	15.2	16.79
2	H002	1390.5	3	4.8	17.54
3	H003	1728.5	4	12.4	19.27
4	H004	2104.9	5	27.0	20.84
5	H005	1349.3	3	2.1	19.76
6	H006	1349.3	3	22.5	14.95
7	H007	2129.1	5	11.4	21.71
8	H008	1780.0	3	4.0	20.81
9	H009	1248.1	3	10.2	17.01
10	H010	1683.3	3	14.9	15.19

Table 2 - Descriptive Statistics of the Features and the Target

Statistic	Area (sq ft)	Bedrooms	Age (years)	Price (Rs lakh)
Count	300	300	300	300
Mean	1449.68	3.21	11.60	16.49
Std. Deviation	417.62	0.93	6.78	3.68
Minimum	500.00	1.00	0.90	7.01
25th Percentile	1156.20	3.00	6.57	13.64
Median (50%)	1475.45	3.00	10.40	16.37
75th Percentile	1719.42	4.00	14.90	19.14
Maximum	3106.70	6.00	36.80	25.49

Table 3 - Training and Testing Errors of the Three Models

S.No	Model	Train RMSE	Test RMSE	Train MAE	Test MAE	Train R^2	Test R^2
1	Linear Regression	1.5636	1.4213	1.2703	1.1148	0.8296	0.8175
2	Decision Tree	0.0000	2.0139	0.0000	1.6139	1.0000	0.6336
3	Polynomial (deg 9)	1.0783	343.0722	0.7115	64.3415	0.9190	-10632.7459

Table 4 - Bias–Variance Diagnosis of every Model

S.No	Model	Test – Train RMSE	Behaviour	Diagnosis
1	Linear Regression	-0.1423	Moderate and nearly equal error on both sets	Slight underfitting (high bias, low variance)
2	Decision Tree	+2.0139	Low training error, clearly higher testing error	Overfitting (high variance)
3	Polynomial (deg 9)	+341.9939	Very low training error, enormous testing error	Severe overfitting (very high variance)

Table 5 - Five-Fold Cross-Validated RMSE of the Three Models

S.No	Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	Std. Deviation
1	Linear Regression	1.417	1.519	1.470	1.660	1.652	1.544	0.097
2	Decision Tree	2.166	2.150	2.454	2.273	2.333	2.275	0.112
3	Polynomial (deg 9)	161.669	246.642	525.270	3082.068	189.205	840.971	1127.977

Table 6 - Actual against Predicted Price for Ten Test Houses

S.No	Actual Price	Linear Regression	Decision Tree	Polynomial (deg 9)	Best Match
1	21.25	19.64	18.03	19.68	Polynomial (deg 9)
2	20.75	21.11	18.60	11.46	Linear Regression
3	14.02	13.76	14.40	14.73	Linear Regression
4	15.19	17.08	17.21	17.69	Linear Regression
5	13.57	12.80	14.75	12.13	Linear Regression
6	14.98	15.58	15.41	14.86	Polynomial (deg 9)
7	13.84	13.61	12.68	14.57	Linear Regression
8	14.77	15.68	13.90	14.37	Polynomial (deg 9)
9	14.95	14.27	14.76	9.89	Decision Tree
10	16.23	16.98	20.87	-24.26	Linear Regression

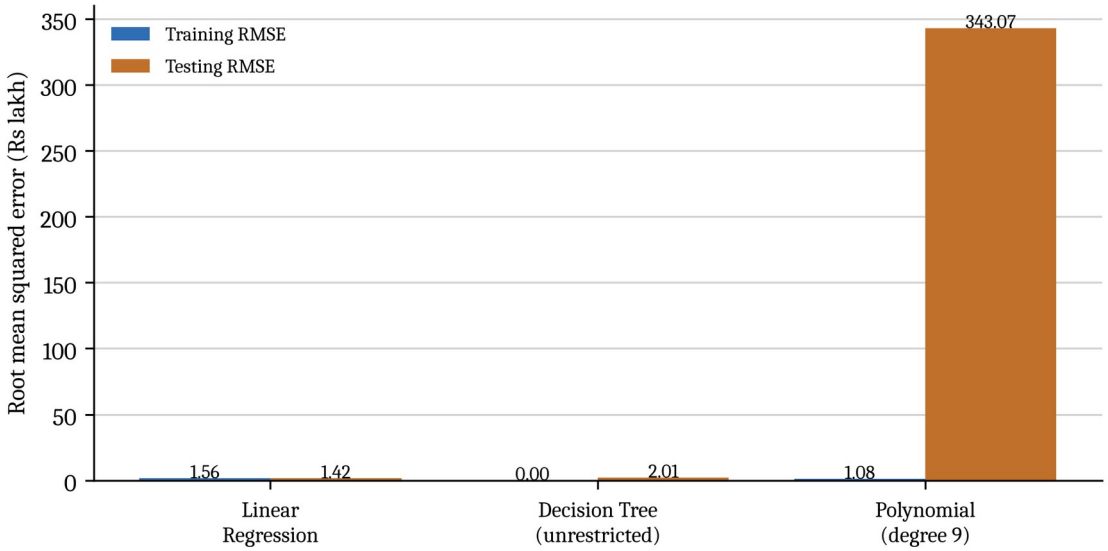
Table 7 - Effect of Model Complexity on the Polynomial Model

Polynomial Degree	Training RMSE	Testing RMSE	Test – Train	Behaviour
1	1.5636	1.4213	-0.1423	Underfitting (high bias)
2	1.5266	1.4136	-0.1130	Good fit
3	1.4877	1.4536	-0.0341	Good fit
4	1.4573	1.5201	+0.0628	Good fit
5	1.3582	2.0941	+0.7359	Overfitting (high variance)
6	1.2468	6.5744	+5.3276	Overfitting (high variance)
7	1.1180	123.5203	+122.4023	Overfitting (high variance)
8	1.0854	55.9499	+54.8645	Overfitting (high variance)
9	1.0783	343.0722	+341.9939	Overfitting (high variance)

Table 8 - Final Evaluation Summary

Parameter	Value
Number of houses	300
Training houses / Testing houses	225 / 75
Features used	Area, Bedrooms, Age
Lowest training RMSE	Decision Tree (0.0000)
Lowest testing RMSE	Linear Regression (1.4213)
Highest testing R^2	Linear Regression (0.8175)
Model with the highest bias	Linear Regression
Overfitting models	Decision Tree (unrestricted), Polynomial (degree 9)
Best polynomial degree observed	2 (test RMSE 1.4136)
Most suitable model	Linear Regression
Reason	Lowest testing RMSE and the smallest train–test gap; its slight bias costs far less than the variance of the other two

Plot 1 - Training and Testing RMSE of the Three Models



Plot 2 - Model Complexity against Error (the Bias–Variance Curve)

